

Anexo A Manual del programador

A 1 Requisitos e inicio

El proyecto requiere Python 3.10 o posterior y un navegador moderno. El backend utiliza únicamente la biblioteca estándar; no se necesita instalar paquetes con pip. La evidencia adjunta se generó con Python 3.12.14 sobre Linux. La ejecución en Windows se ofrece mediante un archivo de inicio y comandos equivalentes.

Extraiga el ZIP completo. Dentro de la carpeta canari, abra una terminal y ejecute:

```
py -3 server.py -demo
```

Abra <http://127.0.0.1:8000>. El servidor muestra una clave de operador; cópiela al formulario del panel y pulse Conectar. Si el comando py no existe, use python después de comprobar la instalación. En Linux o macOS utilice python3. Para detener el proceso, presione Ctrl+C en la terminal.

A 2 Archivos principales

Archivos principales y responsabilidades

Archivo	Responsabilidad
core.py	Validación de muestras, reglas, transacciones y consultas SQLite.
server.py	Rutas HTTP, claves, entrega del panel y supervisor de antigüedad.
config.json	Nodos, ubicación, origen, intervalos y umbrales.
simulator.py	Generación de escenarios y envío por HTTP.
web/index.html	Estructura y controles de la interfaz.
web/app.js y style.css	Consulta, representación visual y estilos.
tests/test_canari.py	Casos automatizados de lógica e integración.
verify.py y backup.py	Evidencia reproducible y respaldo de la base.
firmware/esp32_demo	Ejemplo de envío simulado para ESP32.

A 3 Datos generados al ejecutar

La carpeta data se crea en el primer inicio. Contiene canari.sqlite3, operator.key e ingest.key. Las claves se generan aleatoriamente y no están incluidas en el paquete. La clave de ingreso habilita la recepción de muestras; la de operador permite consultar y reconocer eventos. Las claves no deben incorporarse al código fuente ni compartirse con capturas de pantalla.

Anexo A Contrato de datos y servicios

A 4 Mensaje de telemetría

Cada envío contiene la identidad del nodo, un identificador único de muestra y la fecha de captura con zona horaria. En operación se debe usar la fecha UTC actual. Este ejemplo ilustra el formato:

```
{
  "node_id": "N01",
  "sample_id": "sesionA_001",
  "captured_at": "2026-09-18T04:00:00Z",
  "source": "simulation",
  "device_status": "ok",
  "calibration_ref": "",
  "values": {
    "ch4_vol_pct": 0.2, "o2_vol_pct": 20.9,
    "temperature_c": 24.0, "humidity_pct": 65.0
  }
}
```

Los cuatro campos de values son obligatorios; el valor null representa una medición ausente. No se aceptan booleanos, texto, NaN o infinitos como mediciones. La fecha puede tener hasta 24 horas de antigüedad y como máximo cinco segundos de adelanto. Una fecha aceptada puede, aun así, estar vencida para la vista actual.

A 5 API y autorización

API, claves y funciones

Método y ruta	Clave	Función
GET /api/status	Operador	Estado actual y configuración.
GET /api/readings	Operador	Historial paginado y filtros.
GET /api/events	Operador	Últimos 100 eventos.
POST /api/events/ID/ack	Operador	Registra reconocimiento.
GET /api/export.csv	Operador	CSV de hasta 10 000 registros.
POST /api/telemetry	Ingreso	Recibe una muestra.

Las claves se envían en Authorization con el esquema Bearer. El ingreso de muestras usa Content-Type application/json. Los filtros del historial son node_id, limit, before_id y since. Para continuar, use el next_before_id devuelto; una respuesta sin registros adicionales termina la consulta.

Códigos principales: 201 muestra nueva; 200 consulta o duplicado idéntico; 400 dato inválido; 401 clave incorrecta; 409 identificador repetido con distinto contenido; 413 tamaño no permitido; 415 tipo de contenido incorrecto.

Anexo A Reglas persistencia y mantenimiento

A 6 Flujo de procesamiento

El servidor autentica el emisor y valida estructura, valores, origen y fecha. Después abre una transacción, comprueba duplicados y almacena la lectura con la configuración aplicada. La combinación node_id y sample_id es única: repetir un envío idéntico no crea otro registro; cambiar su contenido genera conflicto.

La lectura vigente se selecciona por fecha de captura. Un mensaje atrasado se conserva para el histórico, pero no reemplaza uno más reciente. La clasificación considera el estado del dispositivo, los límites del nodo y las variables ausentes. Una alarma de gas no queda oculta por la ausencia de otra variable.

El supervisor revisa la antigüedad cada segundo y registra transiciones. La consulta también evalúa si el dato sigue vigente. Al cumplir el tiempo de vencimiento, el nodo muestra Sin datos. Reconocer un evento modifica solo su marca de lectura; no elimina registros ni cambia la condición medida.

A 7 Tablas de la base de datos

Tablas de la base de datos y contenido

Tabla	Contenido
readings	Valores, captura, recepción, origen, calidad declarada, razones y configuración.
events	Cambios de estado, lectura asociada cuando existe y reconocimiento.
node_state	Último estado registrado por nodo para evitar duplicaciones.
configurations	Configuraciones completas identificadas mediante SHA-256.

A 8 Cambios y respaldo

Edite config.json con el servidor detenido y reinícielo. El cambio queda registrado como otra configuración. Conserve los valores anteriores para interpretar el histórico. Para crear una copia coherente de la base, ejecute:

```
py -3 backup.py
```

Para restaurar, detenga el servidor, conserve toda la carpeta data y copie el respaldo como canari.sqlite3 en una carpeta nueva. Arranque con --data-dir seguido de esa carpeta. Las claves se generan si no existen. No sustituya archivos de una base que esté abierta.

El servidor es local y de laboratorio: HTTP no cifra los datos, las claves no representan cuentas individuales y no existe rotación automática. Una migración a producción requiere otra evaluación de despliegue, disponibilidad y controles de acceso (Python Software Foundation, s. f. a).

Anexo A Integración del ejemplo ESP32

A 9 Propósito y componentes

El archivo esp32_demo.ino demuestra el envío de mensajes compatibles con CANARI. Requiere una placa ESP32, Arduino IDE y el core de Espressif. Usa las bibliotecas WiFi, HTTPClient y time incluidas en ese entorno. No contiene controladores de sensores; los cuatro valores transmitidos son simulados.

A 10 Preparación de la comunicación

1. Abra el ejemplo y configure WIFI_SSID, WIFI_PASSWORD, SERVER_URL e INGEST_KEY. La clave es el contenido de data/ingest.key, creado al iniciar el servidor. Sustituya la dirección de ejemplo por la IP privada real del computador.
2. Detenga la simulación automática para evitar dos emisores en N03. Inicie el servidor sin -demo y ligado a la IP del computador. Ejemplo que debe adaptarse a la red:

```
py -3 server.py --host 192.168.1.10
```
3. Conecte computador y placa a una red de laboratorio que permita su comunicación. La placa sincroniza la hora por NTP y no transmite hasta disponer de una fecha válida. No publique el puerto en Internet.
4. Seleccione la placa y el puerto en Arduino IDE, compile y cargue. Revise el monitor serial a 115200 baudios. Una respuesta HTTP 201 indica ingreso nuevo; 401 significa que la clave no coincide. La compilación y el funcionamiento en una placa deben registrarse como evidencia adicional.

A 11 Adaptación a sensores reales

La implementación de un sensor requiere definir su modelo e interfaz: salida digital, serie, bus o señal acondicionada. Deben conservarse sus unidades originales, tiempos de respuesta y señales de fallo. Un valor ADC por sí solo no es una concentración de metano; no se debe reemplazar la simulación por una fórmula arbitraria.

Para un emisor físico, registre un nodo con source igual a laboratory. Use el mismo origen en el mensaje. El estado ok requiere una referencia de calibración declarada; si falta la verificación, use uncalibrated, y ante falla use fault. Esta referencia no valida automáticamente un certificado.

La integración posterior debe evaluarse primero en mesa de laboratorio. La aplicación en una atmósfera explosiva requiere equipos y procedimientos apropiados; una placa de desarrollo común no acredita esas condiciones. El anexo G ofrece una ficha para registrar la evaluación física.